



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Taller No. 4: Modelado de una cuenta bancaria aplicando constructores, sobrecarga y manejo de métodos.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Itzel Barriba Cázares

Marzo/2026

Tabla de contenido

Objetivo.....	3
Lista de materiales	3
Descripción del taller	3
DESARROLLO DE LA PRACTICA.	5
Diagrama UML	5
Clase prueba:	5
Salida en terminal opción 1:	11
Salida en terminal opción 2:	12
Salida en terminal opción 3:	12
Salida en terminal opción 4:	12
Salida en terminal opción 5:	12
Salida en terminal opción 6:	12
Salida en terminal opción 7:	12
Salida en terminal opción 8:	13
Salida en terminal opción 9:	13
Salida en terminal opción 10:	13
Salida en terminal opción 11:	14
Salida en terminal opción 12:	14
Clase Cuenta.....	15
Análisis de resultados	20
Conclusiones	22
Criterios para evaluar:.....	22
Código (70 puntos).....	22
Reporte (30 puntos):.....	22

Objetivo

El alumno diseñará e implementará en Java una clase que modele una Cuenta Bancaria, aplicando de manera correcta los principios fundamentales de la Programación Orientada a Objetos (POO). Al finalizar la práctica, el alumno será capaz de:

- Definir clases con atributos privados aplicando encapsulamiento.
- Implementar constructores sobrecargados para inicializar objetos con distintos niveles de información.
- Aplicar sobrecarga de métodos para ofrecer variantes de una misma operación según los parámetros recibidos.
- Validar datos de entrada dentro de los métodos de la clase, rechazando valores no permitidos.
- Separar la lógica de negocio (clase Cuenta) de la interfaz de usuario (clase Principal), siguiendo el principio de responsabilidad única.

Lista de materiales

- Computadora Personal (PC)
- Ambiente de desarrollo.
- JDK Java Development KitTexto.

Descripción del taller

El alumno diseñará e implementará una clase que modele una Cuenta Bancaria, aplicando correctamente los principios de POO, específicamente:

- Encapsulamiento
- Uso de constructores
- Sobrecarga de constructores
- Sobrecarga de métodos
- Validación de datos
- Separación entre lógica y clase principal.

Diseño UML

Antes de programar, realiza el modelado UML de la clase cuenta

Determina los atributos de la clase, e implementa los métodos para encapsulamiento, depositar(), retirar(), mostrarDatos() y constructores.

El diagrama debe incluir:

- Visibilidad

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

- Tipo de datos
- Parámetros
- Tipo de retorno

Implementar constructores:

- Constructor por defecto (inicializa el saldo en 0),
- Constructor completo (inicializa todos los atributos)
- Constructor sobrecargado (Nombre completo, fecha de nacimiento, saldo inicial)

Manejo de métodos:

Debe existir un método depositar sobrecargado (cantidad) y (cantidad y concepto). No debe permitir

cantidades negativas, mostrar el saldo anterior y el saldo actualizado, si existe concepto, mostrarlo.

El método retirar sobrecargado (cantidad) y (cantidad y concepto). No debe permitir cantidades negativas, no permitir retirar más del saldo disponible. Mostrar mensaje de fondos insuficiente si aplica, mostrar el saldo anterior y el saldo actualizado.

El método Mostrar datos debe devolver un String con nombre completo, dirección completa, fecha de nacimiento, saldo actual.

La clase principal

Crear una clase Principal que:

- Capture los datos mediante menú.
- Solo realice llamadas a métodos
- No contenga lógica bancaria
- Permita repetir operaciones hasta que el usuario seleccione salir.

Menú de Opciones

1. Modificar Nombre
2. Modificar Apellido Paterno
3. Modificar Apellido Materno
4. Modificar Calle
5. Modificar Número
6. Modificar Colonia
7. Modificar Código Postal
8. Modificar Fecha de Nacimiento
9. Depositar
10. Retirar
11. Mostrar datos completos
12. Salir

Cuando se modifique un atributo:

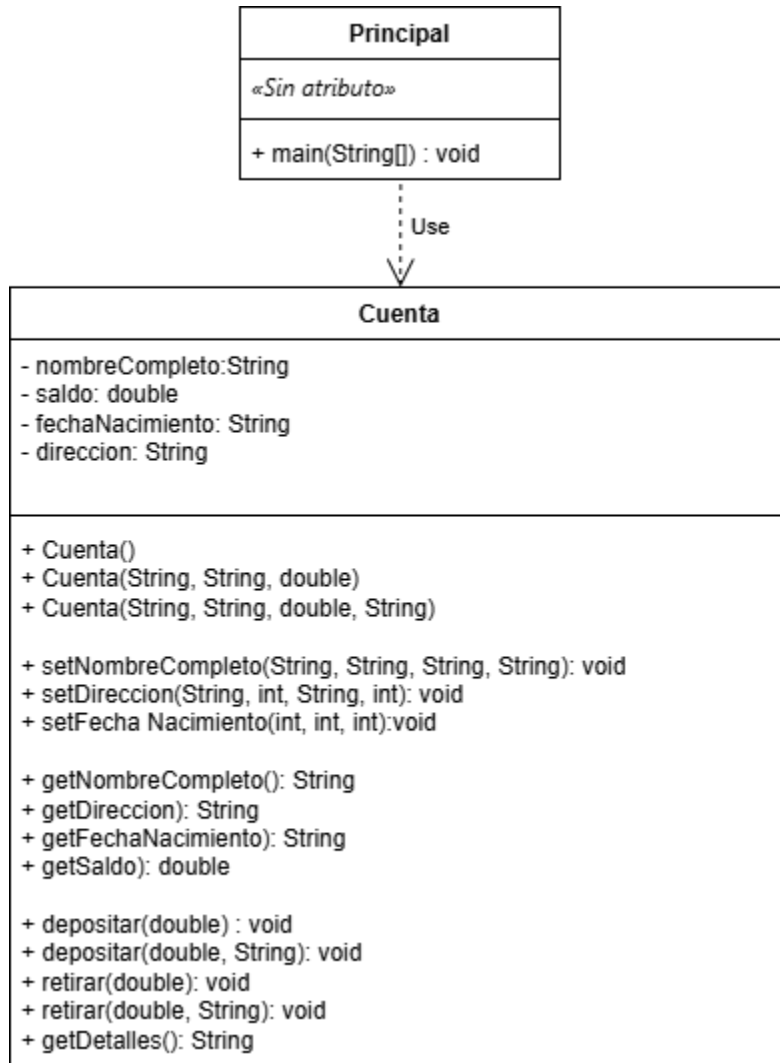
- Mostrar el valor anterior

- Mostrar el nuevo valor.

Adjunte el diagrama de clase y el código fuente, también deberá hacer una demostración de su ejecución en la clase de laboratorio.

DESARROLLO DE LA PRACTICA.

Diagrama UML



Clase prueba:

La clase Principal actúa exclusivamente como interfaz de usuario. No contiene lógica bancaria ni cálculos; su única responsabilidad es capturar datos del usuario y delegar las operaciones a los métodos de la clase Cuenta.

Variables auxiliares:

Se declaran variables locales en main para almacenar de forma temporal los componentes del nombre (primerNombre, segundoNombre, apellidoPaterno, apellidoMaterno) y de la dirección (calle, colonia, numeroCasa, codigoPostal). Esto es necesario porque el setter de nombre y el de dirección requieren todas sus partes en cada llamada para reconstruir el atributo completo. La variable tempStr se reutiliza para capturar respuestas cortas como "S" o "N" y para capturar el concepto de un depósito o retiro.

Menú con do-while y switch:

El menú se implementa con un ciclo do-while que garantiza que las opciones se muestren al menos una vez. Dentro del ciclo, un switch evalúa la opción ingresada y ejecuta el bloque correspondiente. El ciclo se detiene cuando el usuario ingresa la opción 12 (Salir).

Manejo del buffer del Scanner:

Después de cada llamada a input.nextInt() o input.nextDouble(), se agrega input.nextLine() para consumir el salto de línea residual en el buffer. Sin esta instrucción, la siguiente llamada a input.nextLine() leería una cadena vacía en lugar del texto ingresado por el usuario, lo que es un error común al mezclar nextInt con nextLine en Java.

En los casos 5, 7 y 8 (que leen enteros y luego regresan al menú), el input.nextLine() se coloca inmediatamente después del nextInt() para limpiar el buffer antes de que el ciclo solicite la siguiente opción.

```

/*-----
Practica: Taller 5 - Cuenta Bancaria
Nombre: Joshua Osorio
Materia: Programación Orientada a Objetos
Fecha: Marzo del 2026
-----*/

import java.util.Scanner;

public class Principal {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        String primerNombre = "",
            segundoNombre = "",
            apellidoPaterno = "",
            apellidoMaterno = "",
            calle = "",
            colonia = "",
            tempStr = "";

        int dia, mes, anio, numeroCasa = 0, codigoPostal = 0, opcion;
        double monto;

        Cuenta cuenta = new Cuenta();

        do {
            System.out.flush();
            System.out.printf("\n\n\tSistema de Cuenta Bancaria\n");
            System.out.println("\n\t----- Menú -----");
            System.out.println("\t1 - Modificar nombre");
            System.out.println("\t2 - Modificar apellido paterno");
            System.out.println("\t3 - Modificar apellido materno");
            System.out.println("\t4 - Modificar calle");
            System.out.println("\t5 - Modificar número");
            System.out.println("\t6 - Modificar colonia");
            System.out.println("\t7 - Modificar código postal");
            System.out.println("\t8 - Modificar fecha de nacimiento");
            System.out.println("\t9 - Depositar");
            System.out.println("\t10 - Retirar");
            System.out.println("\t11 - Mostrar datos completos");
            System.out.println("\t12 - Salir\n");
            System.out.printf("\tOpción:\t");
            opcion = input.nextInt();
            input.nextLine();

```

```

switch (opcion) {
    case 1:
        System.out.println("\n\n\t\tModificar nombre");
        System.out.printf("\t\tAnterior:\t%s\n", cuenta.getNombreCompleto());
        System.out.printf("\t\t¿Tienes 2 nombres? S / N:\t");
        tempStr = input.nextLine();
        if (tempStr.equalsIgnoreCase("S")) {
            System.out.printf("\t\tIngrese el primer nombre:\t");
            primerNombre = input.nextLine();
            System.out.printf("\t\tIngrese el segundo nombre:\t");
            segundoNombre = input.nextLine();
            cuenta.setNombreCompleto(primerNombre, segundoNombre, apellidoPaterno,
apellidoMaterno);

            System.out.printf("\t\tCambió a:\t%s\n", cuenta.getNombreCompleto());
        } else if (tempStr.equalsIgnoreCase("N")) {
            System.out.printf("\t\tIngrese su nombre:\t");
            primerNombre = input.nextLine();
            segundoNombre = "";
            cuenta.setNombreCompleto(primerNombre, segundoNombre, apellidoPaterno,
apellidoMaterno);

            System.out.printf("\t\tCambió a:\t%s\n", cuenta.getNombreCompleto());
        } else {
            System.out.printf("\t\tOpción no válida.\n");
        }
        break;
    case 2:
        System.out.println("\n\n\t\tModificar apellido paterno");
        System.out.printf("\t\tAnterior:\t%s\n", cuenta.getNombreCompleto());
        System.out.printf("\t\tNuevo apellido paterno:\t");
        apellidoPaterno = input.nextLine();
        cuenta.setNombreCompleto(primerNombre, segundoNombre, apellidoPaterno, ape-
llidoMaterno);

        System.out.printf("\t\tNombre completo cambió a:\t%s\n", cuenta.getNombreCom-
pleto());

        break;
    case 3:
        System.out.println("\n\n\t\tModificar apellido materno");
        System.out.printf("\t\tAnterior:\t%s\n", cuenta.getNombreCompleto());
        System.out.printf("\t\tNuevo apellido materno:\t");
        apellidoMaterno = input.nextLine();
        cuenta.setNombreCompleto(primerNombre, segundoNombre, apellidoPaterno, ape-
llidoMaterno);

        System.out.printf("\t\tNombre completo cambió a:\t%s\n", cuenta.getNombreCom-
pleto());

        break;
    case 4:
        System.out.println("\n\n\t\tModificar calle");
        System.out.printf("\t\tAnterior:\t%s\n", cuenta.getDireccion());
        System.out.printf("\t\tNueva calle:\t");
        calle = input.nextLine();
        cuenta.setDireccion(calle, numeroCasa, colonia, codigoPostal);
        System.out.printf("\t\tCambió a:\t%s\n", calle);
        System.out.printf("\t\tDirección cambió a:\t%s\n", cuenta.getDireccion());
        break;
}

```


UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

```
case 5:
    System.out.println("\n\n\t\tModificar número");
    System.out.printf("\t\tAnterior:\t%s\n", cuenta.getDireccion());
    System.out.printf("\t\tNuevo número de casa:\t");
    numeroCasa = input.nextInt();
    input.nextLine();
    cuenta.setDireccion(calle, numeroCasa, colonia, codigoPostal);
    System.out.printf("\t\tCambió a:\t%d\n", numeroCasa);
    System.out.printf("\t\tDirección cambió a:\t%s\n", cuenta.getDireccion());
    break;

case 6:
    System.out.println("\n\n\t\tModificar colonia");
    System.out.printf("\t\tAnterior:\t%s\n", cuenta.getDireccion());
    System.out.printf("\t\tNueva colonia:\t");
    colonia = input.nextLine();
    cuenta.setDireccion(calle, numeroCasa, colonia, codigoPostal);
    System.out.printf("\t\tCambió a:\t%s\n", colonia);
    System.out.printf("\t\tDirección cambió a:\t%s\n", cuenta.getDireccion());
    break;

case 7:
    System.out.println("\n\n\t\tModificar código postal");
    System.out.printf("\t\tAnterior:\t%s\n", cuenta.getDireccion());
    System.out.printf("\t\tNuevo código postal:\t");
    codigoPostal = input.nextInt();
    input.nextLine();
    cuenta.setDireccion(calle, numeroCasa, colonia, codigoPostal);
    System.out.printf("\t\tCambió a:\t%d\n", codigoPostal);
    System.out.printf("\t\tDirección cambió a:\t%s\n", cuenta.getDireccion());
    break;

case 8:
    System.out.println("\n\n\t\tModificar fecha de nacimiento");
    System.out.printf("\t\tAnterior:\t%s\n", cuenta.getFechaNacimiento());
    System.out.printf("\t\tDía:\t");
    dia = input.nextInt();
    System.out.printf("\t\tMes:\t");
    mes = input.nextInt();
    System.out.printf("\t\tAño:\t");
    anio = input.nextInt();
    input.nextLine();
    cuenta.setFechaNacimiento(dia, mes, anio);
    System.out.printf("\t\tCambió a:\t%s\n", cuenta.getFechaNacimiento());
    break;
```

```

case 9:
    System.out.println("\n\n\t\t----- Depositar -----");
    System.out.printf("\t\tMonto a depositar:\t$");
    monto = input.nextDouble();
    input.nextLine();
    System.out.printf("\t\t¿Agregar descripción? S / N:\t");
    tempStr = input.nextLine();
    if (tempStr.equalsIgnoreCase("S")) {
        System.out.printf("\t\tIngrese descripción:\t");
        tempStr = input.nextLine();
        cuenta.depositar(monto, tempStr);
    } else {
        cuenta.depositar(monto);
    }
    break;
case 10:
    System.out.println("\n\n\t\t----- Retirar -----");
    System.out.printf("\t\tMonto a retirar:\t$");
    monto = input.nextDouble();
    input.nextLine();
    System.out.printf("\t\t¿Agregar descripción? S / N:\t");
    tempStr = input.nextLine();
    if (tempStr.equalsIgnoreCase("S")) {
        System.out.printf("\t\tIngrese descripción:\t");
        tempStr = input.nextLine();
        cuenta.retirar(monto, tempStr);
    } else {
        cuenta.retirar(monto);
    }
    break;
case 11:
    System.out.printf("%s\n", cuenta.getDetalles());
    break;
case 12:
    System.out.printf("\n\tSaliendo...\n");
    break;
default:
    System.out.printf("\n\tOpción no válida, intente nuevamente.\n");
    break;
}

} while (opcion != 12);

input.close();
}
}

```

Captura de lo mostrado en terminal

```
~\OneDrive - uabc.edu.mx\8  x  +  v
> java -cp output Principal

Joe's Automotive Shop

— Menú —
1-      Registrar Cliente
2-      Registrar Automóvil
3-      Generar Cotización
0-      Salir
Opción: |
```

Salida en terminal opción 1:

```
Opción: 1

Modificar nombre
Anterior:      No capturado
¿Tienes 2 nombres? S / N:      s
Ingrese el primer nombre:      Joe
Ingrese el segundo nombre:      Isac
Cambió a:      Joe Isac No capturado No capturado
```

Salida en terminal opción 2:

```
Opción: 2

Modificar apellido paterno
Anterior:      Joe Isac No capturado No capturado
Nuevo apellido paterno: Osorio
Nombre completo cambió a:      Joe Isac Osorio No capturado
```

Salida en terminal opción 3:

```
Opción: 3

Modificar apellido materno
Anterior:      Joe Isac Osorio No capturado
Nuevo apellido materno: Bernal
Nombre completo cambió a:      Joe Isac Osorio Bernal
```

Salida en terminal opción 4:

```
Opción: 4

Modificar calle
Anterior:      No capturado
Nueva calle:    Calzada Universidad
Cambió a:       Calzada Universidad
Dirección cambió a:    Calzada Universidad, No capturado, No capturado, No capturado.
```

Salida en terminal opción 5:

```
Opción: 5

Modificar número
Anterior:      Calzada Universidad, No capturado, No capturado, No capturado.
Nuevo número de casa:    14418
Cambió a:       14418
Dirección cambió a:      Calzada Universidad, 14418, No capturado, No capturado.
```

Salida en terminal opción 6:

```
Opción: 6

Modificar colonia
Anterior:      Calzada Universidad, 14418, No capturado, No capturado.
Nueva colonia: Parque Industrial Otay
Cambió a:       Parque Industrial Otay
Dirección cambió a:    Calzada Universidad, 14418, Parque Industrial Otay, No capturado.
```

Salida en terminal opción 7:

```
Opción: 7

Modificar código postal
Anterior:      Calzada Universidad, 14418, Parque Industrial Otay, No capturado.
Nuevo código postal:    22390
Cambió a:       22390
Dirección cambió a:      Calzada Universidad, 14418, Parque Industrial Otay, 22390.
```

```
Opción: 8

Modificar fecha de nacimiento
Anterior:      No capturado
Dia:          11
Mes:           06
Año:           2002
Cambió a:      11/6/2002
```

[illegible]

———— Retirar ————

Saldo en cuenta 9500.00
Monto a retirar: \$10000
¿Agregar descripción? S / N: n

Fondos insuficientes. Saldo disponible: \$9500.00

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

```
Opción: 10

      Retirar

Saldo en cuenta 9500.00
Monto a retirar:      $1500
¿Agregar descripción? S / N:  s
Ingreso descripción:  Comida china

Saldo anterior: $9500.00
Retiro exitoso.
Concepto:      Comida china
Saldo nuevo:   $8000.00
```

Salida en terminal opción 11:

```
Opción: 11

===== Información de la Cuenta =====
Nombre:      Joe Isac Osorio Bernal
Fecha Nac.:  11/6/2002
Dirección:   Calzada Universidad, 14418, Parque Industrial Otay, 22390.
Saldo actual: $67.00
```

Salida en terminal opción 12:

```
Sistema de Cuenta Bancaria

      Menú

1 - Modificar nombre
2 - Modificar apellido paterno
3 - Modificar apellido materno
4 - Modificar calle
5 - Modificar número
6 - Modificar colonia
7 - Modificar código postal
8 - Modificar fecha de nacimiento
9 - Depositar
10 - Retirar
11 - Mostrar datos completos
12 - Salir

Opción: 12

Saliendo...
Okuyt ~\OneDrive - uabc.edu.mx\8_Semestre\36282_ProgramacinOrientadaAObjetos\3_Taller\T05_Tema main E*?4 ~7 -3 13m 59.52s
```

Clase Cuenta.

La clase Cuenta representa el modelo de una cuenta bancaria. Todos sus atributos son privados, lo que garantiza el encapsulamiento: el estado interno del objeto solo puede ser consultado o modificado a través de los métodos públicos definidos.

Atributos:

- nombreCompleto (String): almacena el nombre completo del titular, construido a partir de sus partes.
- saldo (double): guarda el saldo disponible en la cuenta. Se usa double para permitir valores decimales.
- fechaNacimiento (String): fecha en formato dd/mm/aaaa, construida desde valores enteros porque no hay medios días, meses y años.
- direccion (String): dirección completa del titular, construida a partir de calle, número, colonia y código postal.

Constructores:

Se implementaron tres constructores sobrecargados para cubrir distintos escenarios de inicialización:

- Constructor por defecto Cuenta(): inicializa todos los atributos con el texto "No capturado" y el saldo en 0. Es útil cuando se desea crear el objeto primero y capturar los datos después mediante el menú.
- Constructor sobrecargado Cuenta(nombreCompleto, fechaNacimiento, saldo): permite crear un objeto con el nombre completo del titular, su fecha de nacimiento y un saldo inicial, dejando la dirección como "No capturado".
- Constructor completo Cuenta(nombreCompleto, fechaNacimiento, saldo, direccion): inicializa todos los atributos del objeto en un solo paso, ideal cuando se cuenta con toda la información desde el principio.

Setters compuestos:

- Los setters de esta clase no reciben directamente el String final, sino que construyen los atributos a partir de sus partes:
- setNombreCompleto(nombre1, nombre2, apellidoPaterno, apellidoMaterno): recibe cada parte del nombre de forma independiente. Utiliza el método .isEmpty() para determinar qué partes fueron capturadas y cuáles se marcan como "No capturado". El segundo nombre es opcional y solo se agrega si no está vacío.
- setDireccion(calle, numero, colonia, codigoPostal): construye la cadena de dirección concatenando cada campo, separados por comas. Los campos numéricos se validan contra el valor 0 para detectar si fueron capturados.
- setFechaNacimiento(dia, mes, anio): recibe tres enteros y los concatena en el formato dd/mm/aaaa.

Métodos depositar (sobrecargados):

Existen dos versiones del método depositar, lo que refleja la implementación de sobrecarga de métodos:

- depositar(double cantidad): recibe únicamente el monto a depositar. Primero valida que la cantidad sea mayor a cero; si no lo es, muestra un mensaje de error y termina la ejecución del método con return. Si la cantidad es válida, muestra el saldo anterior, suma la cantidad al saldo y muestra el saldo nuevo.
- depositar(double cantidad, String concepto): funciona igual que la versión anterior, pero además recibe una descripción o concepto del depósito, el cual se muestra en pantalla después de confirmar la operación.

Métodos retirar (sobrecargados):

Para el método depositar, existen dos versiones de retirar:

- retirar(double cantidad): valida primero que la cantidad sea positiva y luego que no supere el saldo disponible. Si alguna validación falla, se muestra el mensaje correspondiente ("Error: No se permiten cantidades negativas o en cero" o "Fondos insuficientes") y se cancela la operación. Si todo es válido, se descuenta el monto del saldo y se muestran los saldos anterior y nuevo.
- retirar(double cantidad, String concepto): idéntico al anterior, pero muestra adicionalmente el concepto del retiro.

Método getDetalles():

Devuelve un String formateado con toda la información de la cuenta: nombre del titular, fecha de nacimiento, dirección completa y saldo actual. Este método centraliza la presentación de datos en la clase Cuenta, manteniendo la lógica de formato dentro del modelo y no en la clase Principal.


```

/*-----
Practica: Taller 5 - Cuenta Bancaria
Nombre: Joshua Osorio
Materia: Programación Orientada a Objetos
-----*/
public class Cuenta {

    // -----
    // Atributos privados (Encapsulamiento)
    // -----
    private String nombreCompleto;
    private double saldo;
    private String fechaNacimiento;
    private String direccion;

    // -----
    // Constructor por defecto - saldo en 0, datos vacíos
    // -----
    public Cuenta() {
        this.nombreCompleto = "No capturado";
        this.saldo = 0;
        this.fechaNacimiento = "No capturado";
        this.direccion = "No capturado";
    }

    // -----
    // Constructor sencillo - nombre, fecha y saldo inicial
    // -----
    public Cuenta(String nombreCompleto, String fechaNacimiento, double saldo) {
        this.nombreCompleto = nombreCompleto;
        this.fechaNacimiento = fechaNacimiento;
        this.saldo = saldo;
        this.direccion = "No capturado";
    }

    // -----
    // Constructor completo - todos los atributos
    // -----
    public Cuenta(String nombreCompleto, String fechaNacimiento, double saldo, String direccion)
    {
        this.nombreCompleto = nombreCompleto;
        this.fechaNacimiento = fechaNacimiento;
        this.saldo = saldo;
        this.direccion = direccion;
    }
}

```

```

// -----
// Setters
// -----
public void setNombreCompleto(String nombre1, String nombre2,
    String apellidoPaterno, String apellidoMaterno) {
    nombreCompleto = "";

    if (!nombre1.isEmpty()) {
        nombreCompleto += nombre1 + " ";
    } else {
        nombreCompleto += "No capturado ";
    }

    if (!nombre2.isEmpty()) {
        nombreCompleto += nombre2 + " ";
    }

    if (!apellidoPaterno.isEmpty()) {
        nombreCompleto += apellidoPaterno + " ";
    } else {
        nombreCompleto += "No capturado ";
    }

    if (!apellidoMaterno.isEmpty()) {
        nombreCompleto += apellidoMaterno;
    } else {
        nombreCompleto += "No capturado";
    }
}

public void setDireccion(String calle, int numero, String colonia, int codigoPostal) {
    direccion = "";
    if (!calle.isEmpty()) {
        direccion += calle + ", ";
    } else {
        direccion += "No capturado, ";
    }
    if (numero != 0) {
        direccion += numero + ", ";
    } else {
        direccion += "No capturado, ";
    }
    if (!colonia.isEmpty()) {
        direccion += colonia + ", ";
    } else {
        direccion += "No capturado, ";
    }
    if (codigoPostal != 0) {
        direccion += codigoPostal + ".";
    } else {
        direccion += "No capturado.";
    }
}
}

```

```

public void setFechaNacimiento(int dia, int mes, int anio) {
    this.fechaNacimiento = dia + "/" + mes + "/" + anio;
}

// -----
// Getters
// -----
public String getNombreCompleto() {
    return nombreCompleto;
}

public String getDireccion() {
    return direccion;
}

public String getFechaNacimiento() {
    return fechaNacimiento;
}

public double getSaldo() {
    return saldo;
}

// -----
// depositar - sin concepto
// -----
public void depositar(double cantidad) {
    if (cantidad <= 0) {
        System.out.printf("\n\t\tError: No se permiten cantidades negativas o en cero.\n");
        return;
    }
    System.out.printf("\n\t\tSaldo anterior:\t$%.2f\n", saldo);
    saldo += cantidad;
    System.out.printf("\t\tDeposito exitoso.\n");
    System.out.printf("\t\tSaldo nuevo:\t$%.2f\n", saldo);
}

// -----
// depositar - con concepto
// -----
public void depositar(double cantidad, String concepto) {
    if (cantidad <= 0) {
        System.out.printf("\n\t\tError: No se permiten cantidades negativas o en cero.\n");
        return;
    }
    System.out.printf("\n\t\tSaldo anterior:\t$%.2f\n", saldo);
    saldo += cantidad;
    System.out.printf("\t\tDeposito exitoso.\n");
    System.out.printf("\t\tConcepto:\t%s\n", concepto);
    System.out.printf("\t\tSaldo nuevo:\t$%.2f\n", saldo);
}

```

```
// -----  
// retirar - sin concepto  
// -----  
public void retirar(double cantidad) {  
    if (cantidad <= 0) {  
        System.out.printf("\n\t\tError: No se permiten cantidades negativas o en cero.\n");  
        return;  
    }  
    if (cantidad > saldo) {  
        System.out.printf("\n\t\tFondos insuficientes. Saldo disponible: $%.2f\n", saldo);  
        return;  
    }  
    System.out.printf("\n\t\tSaldo anterior:\t$%.2f\n", saldo);  
    saldo -= cantidad;  
    System.out.printf("\t\tRetiro exitoso.\n");  
    System.out.printf("\t\tSaldo nuevo:\t$%.2f\n", saldo);  
}  
  
// -----  
// retirar - con concepto  
// -----  
public void retirar(double cantidad, String concepto) {  
    if (cantidad <= 0) {  
        System.out.printf("\n\t\tError: No se permiten cantidades negativas o en cero.\n");  
        return;  
    }  
    if (cantidad > saldo) {  
        System.out.printf("\n\t\tFondos insuficientes. Saldo disponible: $%.2f\n", saldo);  
        return;  
    }  
    System.out.printf("\n\t\tSaldo anterior:\t$%.2f\n", saldo);  
    saldo -= cantidad;  
    System.out.printf("\t\tRetiro exitoso.\n");  
    System.out.printf("\t\tConcepto:\t%s\n", concepto);  
    System.out.printf("\t\tSaldo nuevo:\t$%.2f\n", saldo);  
}  
  
// -----  
// getDetalles() - devuelve String con todos los datos  
// -----  
public String getDetalles() {  
    return "\n\t\t==== Información de la Cuenta =====" +  
        "\n\t\t\tNombre:\t\t" + nombreCompleto +  
        "\n\t\t\tFecha Nac.:\t" + fechaNacimiento +  
        "\n\t\t\tDirección:\t" + direccion +  
        "\n\t\t\tSaldo actual:\t$" + String.format("%.2f", saldo) +  
        "\n\t\t\t=====";  
}  
}
```

Análisis de resultados

Opciones 1 al 3 — Modificación del nombre:

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

Al ingresar la opción 1, el sistema pregunta si el usuario tiene dos nombres. En la primera captura se ingresaron "Joe" e "Isac" como primer y segundo nombre respectivamente; el sistema los concatenó correctamente mostrando "Joe Isac No capturado No capturado", ya que los apellidos aún no habían sido capturados. Posteriormente, con las opciones 2 y 3 se ingresaron el apellido paterno "Osorio" y el materno "Bernal", actualizando el nombre completo a "Joe Isac Osorio Bernal". Esto confirma que el método `setNombreCompleto` reconstruye correctamente el nombre completo con cada modificación parcial.

Opciones 4 al 7 — Modificación de la dirección:

El sistema construye la dirección de forma progresiva. Al ingresar únicamente la calle "Calzada Universidad", la dirección se mostró como "Calzada Universidad, No capturado, No capturado, No capturado.", confirmando que los campos pendientes se marcan correctamente. Conforme se agregaron el número (14418), la colonia ("Parque Industrial Otay") y el código postal (22390), la dirección se completó hasta mostrar "Calzada Universidad, 14418, Parque Industrial Otay, 22390.", lo cual valida el funcionamiento del método `setDireccion`.

Opción 8 — Modificación de fecha de nacimiento:

Se ingresó el día 11, mes 06 y año 2002. El sistema formateó y mostró correctamente la fecha como "11/6/2002", verificando el funcionamiento del método `setFechaNacimiento`.

Opción 9 — Depósitos:

Se realizaron varias pruebas: un depósito de \$1,500 sin concepto fue aceptado correctamente, actualizando el saldo de \$0.00 a \$1,500.00. Un segundo depósito de \$67 con el concepto "Para la pizza" también fue procesado, mostrando el concepto en pantalla y actualizando el saldo a \$1,567.00. Se probaron además casos de validación: al ingresar -\$1,500 y \$0, el sistema rechazó ambos con el mensaje "Error: No se permiten cantidades negativas o en cero", lo que confirma que la validación funciona correctamente. Un monto con muchos decimales como \$0.999... fue redondeado a \$1.00 por el formato `%.2f` al mostrarse.

Opción 10 — Retiros:

Al intentar retirar \$10,000 con un saldo de \$9,500.00, el sistema respondió con "Fondos insuficientes. Saldo disponible: \$9,500.00", confirmando la validación de fondos. En una segunda prueba se retiró \$1,500 con el concepto "Comida china", operación que fue exitosa y actualizó el saldo a \$8,000.00.

Opción 11 — Mostrar datos completos:

La salida mostró correctamente todos los campos del objeto: nombre "Joe Isac Osorio Bernal", fecha de nacimiento "11/6/2002", dirección completa y saldo actual de \$67.00 (saldo resultante de los depósitos y retiros previos en esa sesión). Esto confirma que el método `getDetalles()` recopila y presenta correctamente toda la información de la cuenta.

Opción 12 — Salir:

Al seleccionar la opción 12, el sistema mostró "Saliendo..." y terminó la ejecución, lo que confirma que la condición del ciclo `do-while` funciona correctamente.

Conclusiones

La práctica permitió comprender de forma práctica y aplicada los principios fundamentales de la Programación Orientada a Objetos. A través del modelado de una cuenta bancaria, se consolidaron los siguientes aprendizajes: Encapsulamiento, sobre carga de constructores, sobrecarga de métodos, validación de datos y separación de responsabilidades.

Criterios para evaluar:

Código (70 puntos)

- Debe incluir comentarios en su programa, sin llegar al uso exagerado de ellos.
- Debe utilizar sangrías en los lugares apropiados.
- Poner como encabezado del programa en comentarios, título, la descripción de la práctica, nombre del alumno que realizó la práctica.
- Funcionamiento correcto del programa

Reporte (30 puntos):

Portada, competencia u objetivo	2.5 puntos
Pegar Código del programa (anexar archivos .py)	2.5 puntos
Explicación completa del código y su funcionamiento	15 puntos
Resultados (análisis de resultados)	5 puntos
Conclusiones:	5 puntos

Nota: No se aceptarán trabajos fuera de la fecha de entrega o que no cumplan con las especificaciones de entrega requeridas. Los trabajos que se detecten copiados serán anulados automáticamente para ambos alumnos.